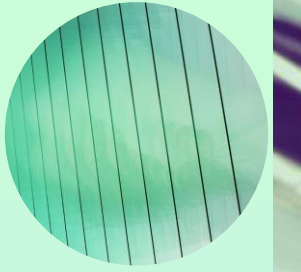


Online ATM System



Abdul Rafay 2280101



Introduction

The Online ATM System significantly enhances banking convenience by enabling users to carry out a wide range of financial transactions through a physical ATM machine. This innovative system offers secure and efficient financial management, allowing users to access their accounts and perform transactions anytime and anywhere, streamlining the banking experience beyond traditional interactions.



Main Features

Account Creation

Personal information like;
name, age, city

Account Operations

Withdraw
Transfer
Deposit
Check Balance
Change Pin
Transaction History
Block Card
Unblock Card
Apply for Loan
Exit to Main Menu



Description

Java Variables	Loops (while, for)	Objects	Methods (Accessor, Mutator)	Conditional Statements	Abstraction
Data Types	Array	Fields	Parameters	Scanner	Inheritance
Typecasting (Widening)	Classes	Constructor	Assignment	Polymorphism (Method Overriding)	Encapsulation

Account Creation

```
while (true) {
    System.out.println("\nWELCOME TO THE ONLINE ATM SYSTEM");
    System.out.println("1: Create a new account");
    System.out.println("2: ATM operations");
    System.out.println("0: Exit");
    System.out.print("Please select an option (1-2, 0 to exit): ");
    int a = scanner.nextInt(); // Parameter and Assignment
    switch (a) {
        case 1: // Account creation
            System.out.println("You have selected account creation");
            System.out.print("Enter your first name: ");
            String firstName = scanner.next();
            System.out.print("Enter your last name: ");
            String lastName = scanner.next();
            System.out.print("Enter your city: ");
            String city = scanner.next();
            System.out.print("Enter your age: ");
            int age = scanner.nextInt();
            if (age < 18) {
                System.out.println("You are underage for creating an account. Must be at least 18 years old.");
                break;
            }
            System.out.print("Enter your initial balance: ");
            double initialBalance = scanner.nextDouble();
            int pin;
            while (true) {
                System.out.print("Enter a unique 4-digit pin: ");
                pin = scanner.nextInt();
                // Check if the pin is already used
                if (!usedPins.contains(pin)) {
                    usedPins.add(pin);
                    break; // Exit loop if the PIN is unique
                } else {
                    System.out.println("This PIN is already taken. Please choose a different PIN.");
                }
            }
        case 2: // ATM operations
            // ...
        case 0: // Exit
            // ...
    }
}
```

Output

```
WELCOME TO THE ONLINE ATM SYSTEM
1: Create a new account
2: ATM operations
0: Exit
Please select an option (1-2, 0 to exit): 1
Account creation
Enter first name: aman
Enter last name: saad
Enter city: isb
Enter age: 19
Enter initial balance: 10000
Enter a unique 4-digit pin: 1234
Account created. Account Number: 8.590715144861993E8

WELCOME TO THE ONLINE ATM SYSTEM
1: Create a new account
2: ATM operations
0: Exit
Please select an option (1-2, 0 to exit): 1
Account creation
Enter first name: rafay
Enter last name: miraj
Enter city: lhr
Enter age: 20
Enter initial balance: 15000
Enter a unique 4-digit pin: 2345
Account created. Account Number: 8.958359821274549E8
```


Account Operations

Withdraw (Polymorphism – Overriding)

Output

```
// Withdraw method
@Override
public void withdraw(double amount) {
    if (isCardBlocked) {
        System.out.println("Transaction declined. Your card is blocked.");
        return;
    }

    if (amount <= balance) {
        balance -= amount;
        transactionHistory.add("Withdraw: Rs" + amount);
        System.out.println("Please collect your cash: Rs" + amount);
        System.out.println("Remaining balance: Rs" + balance);
    } else {
        System.out.println("Insufficient balance.");
    }
}
```

```
Select an Option:
1: Withdraw
2: Transfer
3: Deposit
4: Check Balance
5: Change PIN
6: View Transaction History
7: Block Card
8: Unblock Card
9: Apply for Loan
0: Exit to Main Menu
1
Enter Amount: 1500
Please collect your cash: Rs1500.0
Remaining balance: Rs8500.0
Do you want to continue? (yes/no): yes
```

Transfer

```
// Transfer method
public void transfer(OnlineATMSystem recipient, double amount) {
    if (isCardBlocked) {
        System.out.println("Transaction declined. Your card is blocked.");
        return;
    }

    if (amount <= balance) {
        balance -= amount;
        recipient.deposit(amount);
        transactionHistory.add("Transfer: Rs" + amount + " to Account " + recipient.getAccountNumber());
        System.out.println("Amount Transferred: Rs" + amount);
        System.out.println("Remaining balance: Rs" + balance);
    } else {
        System.out.println("Insufficient balance.");
    }
}
```

Output

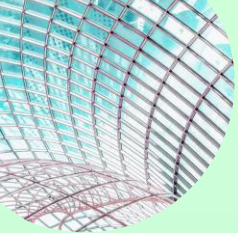
```
Select an Option:
1: Withdraw
2: Transfer
3: Deposit
4: Check Balance
5: Change PIN
6: View Transaction History
7: Block Card
8: Unblock Card
9: Apply for Loan
0: Exit to Main Menu
2
Enter Recipient's Account Number: 8.590715144861993E8
Enter Amount: 1640
Amount Deposited: Rs1640.0
Updated balance: Rs8500.0
Amount Transferred: Rs1640.0
Remaining balance: Rs8500.0
Do you want to continue? (yes/no): yes
```

Deposit (Polymorphism – Overriding)

```
public void deposit(double amount) {  
    balance += amount;  
    transactionHistory.add("Deposit: Rs" + amount);  
    System.out.println("Amount Deposited: Rs" + amount);  
    System.out.println("Updated balance: Rs" + balance);  
}
```

Output

```
Select an Option:  
1: Withdraw  
2: Transfer  
3: Deposit  
4: Check Balance  
5: Change PIN  
6: View Transaction History  
7: Block Card  
8: Unblock Card  
9: Apply for Loan  
0: Exit to Main Menu  
3  
Enter Amount to Deposit: 2000  
Amount Deposited: Rs2000.0  
Updated balance: Rs10500.0  
Do you want to continue? (yes/no): yes
```

Check Balance

```
public double getBalance() {  
    return balance;  
}
```

Change Pin

```
public void changePin(int newPin) {  
    this.pin = newPin;  
    System.out.println("PIN successfully changed.");  
}
```

Output

Select an Option:

1: Withdraw
2: Transfer
3: Deposit
4: Check Balance
5: Change PIN
6: View Transaction History
7: Block Card
8: Unblock Card
9: Apply for Loan
0: Exit to Main Menu

4

Current Balance: Rs10500.0

Do you want to continue? (yes/no): yes

Select an Option:

1: Withdraw
2: Transfer
3: Deposit
4: Check Balance
5: Change PIN
6: View Transaction History
7: Block Card
8: Unblock Card
9: Apply for Loan
0: Exit to Main Menu

5

Enter New PIN: 2978

PIN successfully changed.

Do you want to continue? (yes/no): yes

Transaction History

```
public ArrayList<String> getTransactionHistory() {  
    return transactionHistory;  
}
```

Block Card

```
// Block Card method  
public void blockCard(String reason) {  
    isCardBlocked = true;  
    System.out.println("Your card has been temporarily blocked. Reason: " + reason);  
}
```

Output

Select an Option:

1: Withdraw
2: Transfer
3: Deposit
4: Check Balance
5: Change PIN
6: View Transaction History
7: Block Card
8: Unblock Card
9: Apply for Loan
0: Exit to Main Menu

6

Transaction History:

Withdraw: Rs1500.0

Deposit: Rs1640.0

Transfer: Rs1640.0 to Account 8.590715144861993E8

Deposit: Rs2000.0

Do you want to continue? (yes/no): yes

Select an Option:

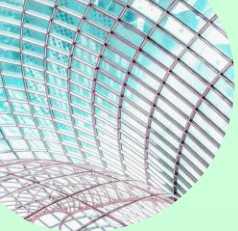
1: Withdraw
2: Transfer
3: Deposit
4: Check Balance
5: Change PIN
6: View Transaction History
7: Block Card
8: Unblock Card
9: Apply for Loan
0: Exit to Main Menu

7

Enter reason for blocking: stolen

Your card has been temporarily blocked. Reason: stolen

Do you want to continue? (yes/no): yes



Unlock Card

```
// Unlock Card method
public void unlockCard(String name, double accNum, int pin) {
    if (pin == this.pin) {
        isCardBlocked = false;
        System.out.println("Your card has been unlocked.");
    } else {
        System.out.println("Invalid PIN. Unable to unlock the card.");
    }
}
```

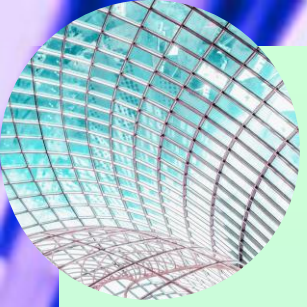
Apply for Loan

```
// Loan management feature
public void applyForLoan(double loanAmount) {
    System.out.println("Reminder: You need to pay off the loan in time.");
    System.out.println("How do you plan to repay the loan?");
    System.out.println("1: Within 15 days");
    System.out.println("2: Within 1 month");
    System.out.println("3: Within 1 year");
    Scanner scanner = new Scanner(System.in);
    int choice = scanner.nextInt();
}
```

Output

```
2: Transfer
3: Deposit
4: Check Balance
5: Change PIN
6: View Transaction History
7: Block Card
8: Unblock Card
9: Apply for Loan
0: Exit to Main Menu
8
Enter your name: aman
Enter your account number: 8.590715144861993E8
Enter your PIN: 2978
Your card has been unlocked.
Do you want to continue? (yes/no): yes

Select an Option:
1: Withdraw
2: Transfer
3: Deposit
4: Check Balance
5: Change PIN
6: View Transaction History
7: Block Card
8: Unblock Card
9: Apply for Loan
0: Exit to Main Menu
9
Enter loan amount: 57000
Reminder: You need to pay off the loan in time.
How do you plan to repay the loan?
1: Within 15 days
2: Within 1 month
3: Within 1 year
2
You have chosen to repay the loan within 1 month.
Loan application received for amount: Rs57000.0
Do you want to continue? (yes/no): no
```

Encapsulation

Data members like `accountNumber`, `balance`, `pin`, etc., are encapsulated in the `BankAccount` class with protected access, and are only accessible and modifiable through specific methods (e.g., `withdraw`, `deposit`, `getBalance`).

The benefit of using these accessor methods is that they provide a way to retrieve field values without allowing direct access to the fields themselves, enhancing security and maintainability of the code.

```
protected double accountNumber;  
protected String firstName;  
protected String lastName;  
protected String city;  
protected double balance;  
protected int pin;  
protected boolean isCardBlocked;  
protected ArrayList<String> transactionHistory;  
protected ArrayList<String> personalizedAlerts;
```

```
public double getAccountNumber() {  
    return accountNumber;  
}  
public double getBalance() {  
    return balance;  
}  
public int getPin() {  
    return pin;  
}  
public ArrayList<String> getTransactionHistory() {  
    return transactionHistory;  
}
```

```
public String getCity() {  
    return city;  
}  
public boolean isCardBlocked() {  
    return isCardBlocked;  
}  
public ArrayList<String> getPersonalizedAlerts() {  
    return personalizedAlerts;  
}  
public void addAlert(String alert) {  
    personalizedAlerts.add(alert);  
}
```

Abstraction

The BankAccount class is declared as abstract and defines abstract methods like withdraw and deposit, which are implemented in the OnlineATMSystem subclass.

Abstraction is useful because it simplifies the design and keeps only the necessary actions visible, while details are handled in specific subclasses.

```
// Abstract base class (Abstraction)
abstract class BankAccount {
    protected double accountNumber;
    protected String firstName;
    protected String lastName;
    protected String city;
    protected double balance;
    protected int pin;
    protected boolean isCardBlocked;
    protected ArrayList<String> transactionHistory;
    protected ArrayList<String> personalizedAlerts;
```

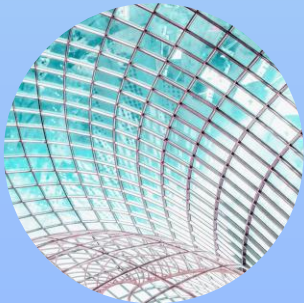
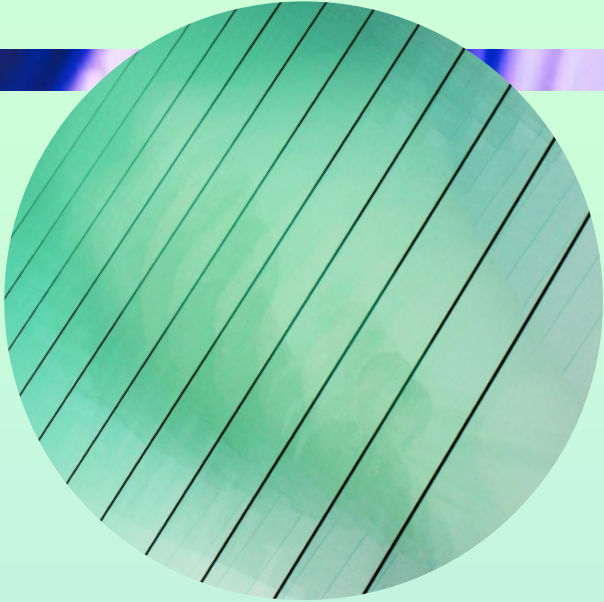
```
// Abstract methods (Abstraction)
public abstract void withdraw(double amount);
public abstract void deposit(double amount);
```

Inheritance

Inheritance allows a class to inherit the fields and methods of another class. In this code, OnlineATMSystem inherits from BankAccount, meaning it can use the properties and methods defined in BankAccount. Additionally, it overrides abstract methods from the BankAccount class to provide specific implementations for withdraw and deposit.

```
// OnlineATMSystem inherits BankAccount (Inheritance)
public class OnlineATMSystem extends BankAccount {

    // Constructor (Inheritance)
    public OnlineATMSystem(double accountNumber, String firstName, String lastName, String city, double initialBalance, int pin)
    {
        super(accountNumber, firstName, lastName, city, initialBalance, pin); // Call to parent class constructor
    }
}
```

Thank you!